however the function exits. Note that the calls to `futures[i].get()` ❷ will block until the results are ready, so you don't need to have explicitly joined with the threads at this point. This is unlike the original from listing 8.2, where you needed to have joined with the threads to ensure that the `results` vector was correctly populated. Not only do you get exception-safe code, but your function is actually shorter because you've extracted the join code into your new (reusable) class.

Exception Safety with Std::Async()

Now that you've seen what's required for exception safety when explicitly managing the threads, let's take a look at the same thing done with `std::async()`. As you've already seen, in this case the library takes care of managing the threads for you, and any threads spawned are completed when the future is *ready*. The key thing to note for exception safety is that if you destroy the future without waiting for it, the destructor will wait for the thread to complete. This neatly avoids the problem of leaked threads that are still executing and holding references to the data. The next listing shows an exception-safe implementation using `std::async()`.

Listing 8.5. An exception-safe parallel version of `std::accumulate` using `std::async`

```
template<typename Iterator,typename T>
T parallel_accumulate(Iterator first,Iterator last,T init)
{
    unsigned long const length=std::distance(first,last);     <-❶
    unsigned long const max_chunk_size=25;
    if(length<=max_chunk_size)
    {
        return std::accumulate(first,last,init);     <-❷
    }
    else
    {
    Iterator mid_point=first;
        std::advance(mid_point,length/2);     <-❸
        std::future<T> first_half_result=
            std::async(parallel_accumulate<Iterator,T>,     <-❹
                       first,mid_point,init);
        T second_half_result=parallel_accumulate(mid_point,last,T());     <-❺
        return first_half_result.get()+second_half_result;     <-❻
    }
}
```